

DOCUMENTACIÓN TÉCNICA OFICIAL



MinkOracle

Motor de Inteligencia de Enjambre Conciso y Universal

Versión

v0.1-Preview

Licencia

AGPL-3.0

Stack

Flask · Vue 3 · OASIS · Zep

Fecha

Junio 2026

🔗 github.com/tatanGod17/mirofish

Tabla de Contenidos

Guía de navegación del documento

1	Visión General del Proyecto	3
2	Arquitectura del Sistema	4
3	Servicios de IA & LLM Utilizados	5
4	Configuración de Credenciales	6
5	Pipeline de 5 Pasos – Detalle Técnico	8
6	Stack Tecnológico Completo	11
7	Estructura de Archivos del Proyecto	12
8	API REST – Referencia de Endpoints	13
9	Comandos para Ejecutar el Proyecto	14
10	Repositorio & Recursos	15

5

Pasos del Pipeline

3

APIs de IA Integradas

3

Idiomas Soportados

1. Visión General del Proyecto

MinkOracle (anteriormente MiroFish) es un motor de simulación de inteligencia de enjambre que permite predecir el comportamiento colectivo de agentes en redes sociales. A partir de un único documento (PDF, Markdown o texto), el sistema construye automáticamente un grafo de conocimiento, genera perfiles de agentes realistas y simula su interacción en plataformas como Twitter y Reddit.

Propuesta de valor: Sube un documento de contexto → MinkOracle genera hasta millones de agentes IA → simula sus decisiones en redes sociales → produce reportes predictivos accionables.

¿Qué puede hacer MinkOracle?

- **Análisis de documentos:** Extrae automáticamente entidades, relaciones y ontología de cualquier documento corporativo o de investigación.
- **Construcción de grafos:** Crea un Knowledge Graph enriquecido en Zep Cloud con todas las entidades y sus relaciones.
- **Generación de agentes:** Convierte entidades del grafo en perfiles de agentes IA con personalidades, historiales y comportamientos únicos.
- **Simulación de redes sociales:** Ejecuta la simulación en plataformas Twitter/Reddit con el framework OASIS/CAMEL-AI.
- **Reportes inteligentes:** El ReportAgent utiliza razonamiento ReACT para generar análisis profundos con insights accionables.
- **Chat interactivo:** Interactúa con el ReportAgent para hacer preguntas de seguimiento sobre los resultados.

Casos de Uso Principales



Análisis de Opinión Pública

Predice cómo reaccionará el público ante lanzamientos de productos, campañas o eventos usando agentes que simulan comportamiento real.



Investigación Social

Estudia dinámicas grupales complejas: propagación de información, formación de opiniones, detección de cámaras de eco.



Marketing Predictivo

Optimiza estrategias de comunicación antes de ejecutarlas, encontrando el "óptimo local" entre diferentes enfoques.

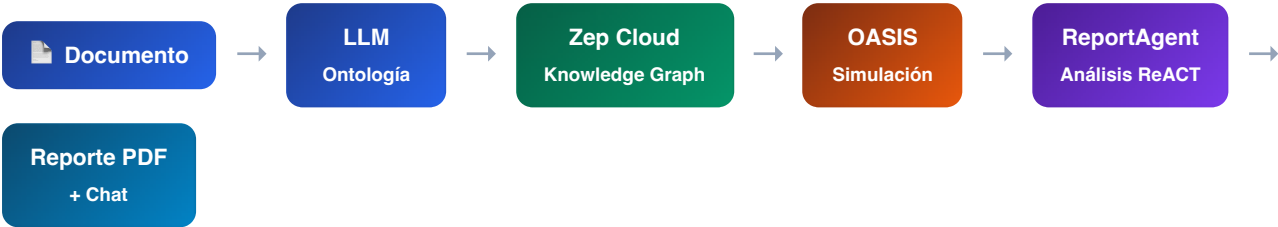


Análisis de Crisis

Simula escenarios de crisis para anticipar reacciones y preparar respuestas estratégicas con antelación.

2. Arquitectura del Sistema

Flujo de Datos General



Componentes Principales

Capa	Tecnología	Puerto / URL	Responsabilidad
Frontend	Vue 3 + Vite + D3.js	localhost:3000	UI interactiva de las 5 fases del pipeline
Backend API	Flask 3.0 + Python 3.11+	localhost:5001	Orquestador REST de todos los servicios
Knowledge Graph	Zep Cloud API	Cloud (API Key)	Almacena y consulta entidades y relaciones
LLM Gateway	OpenAI SDK (formato compatible)	Configurable via .env	Generación de texto, análisis, razonamiento
Simulación	OASIS + CAMEL-AI	Subprocess local	Motor de simulación multi-agente en redes sociales

Blueprints de la API REST

 **/api/graph**

Gestión de proyectos, generación de ontología y construcción del Knowledge Graph en Zep.

graph_bp project management

 **/api/simulation**

Preparación de perfiles de agentes, configuración de simulación, inicio/parada y monitoreo en tiempo real.

simulation_bp async execution

 **/api/report**

 **Almacenamiento Local**

Generación de reportes con ReportAgent,
exportación PDF y chat interactivo con el agente.

report_bp

ReACT pattern

Todos los datos se persisten en JSON/JSONL en
`backend/uploads/` — sin base de datos
externa.

projects/

simulations/

reports/

3. Servicios de IA & LLM Utilizados



MinkOracle usa el **formato OpenAI SDK** como interfaz unificada. Esto permite conectar cualquier proveedor compatible (Alibaba Qwen, OpenAI GPT, Deepseek, Mistral, etc.) simplemente cambiando las variables de entorno.

Proveedor LLM Principal — Alibaba Qwen (Recomendado)



Alibaba Cloud Bailian — Modelo: qwen-plus

Proveedor recomendado por costo-eficiencia. Soporta el formato OpenAI SDK directamente.

- **URL base:** `https://dashscope.aliyuncs.com/compatible-mode/v1`
- **Modelos recomendados:** `qwen-plus`, `qwen-turbo`, `qwen-max`
- **Consola:** `https://bailian.console.aliyun.com/`
- **Nota:** Las simulaciones consumen bastante — comenzar con menos de 40 rondas.

Proveedor LLM Alternativo — OpenAI



OpenAI — Modelos GPT-4o / GPT-4o-mini

Compatible directamente. Solo cambiar la API Key y dejar la URL vacía (usa el default de OpenAI).

- **URL base:** `https://api.openai.com/v1` (default del SDK)
- **Modelos:** `gpt-4o`, `gpt-4o-mini`, `o1-mini`

Zep Cloud — Knowledge Graph & Memoria



Zep Cloud — Motor de Grafo de Conocimiento

Servicio central para almacenar, enriquecer y consultar el grafo de entidades. El tier gratuito mensual soporta casos de uso simples.

- **Función en Step 2:** Construye el Knowledge Graph con ontología definida en Step 1
- **Función en Step 3:** Lee entidades filtradas para generar perfiles de agentes
- **Función en Step 5:** ReportAgent usa Zep para búsqueda híbrida (InsightForge, PanoramaSearch)
- **Registro:** `https://app.getzep.com/`

OASIS + CAMEL-AI — Motor de Simulación

⚡ OASIS (camel-oasis 0.2.5) + CAMEL-AI (camel-ai 0.2.78)

Framework de código abierto para simulación multi-agente en redes sociales. Cada agente simula comportamiento en Twitter y/o Reddit.

- **Plataformas simuladas:** Twitter (tweets, likes, retweets, follows) y Reddit (posts, comments, upvotes)
- **Escalabilidad:** Soporta desde decenas hasta millones de agentes (depende de recursos)
- **No requiere API keys:** Simulación completamente local

Herramientas del ReportAgent (Zep Tools)

Herramienta	Función	Cuándo se usa
InsightForge	Búsqueda híbrida profunda con sub-preguntas generadas por LLM y recuperación multi-dimensional	Análisis principal de cada sección del reporte
PanoramaSearch	Búsqueda amplia incluyendo datos expirados en el grafo de Zep	Contexto histórico y datos de largo plazo
Interview	Conversación directa con agentes individuales del grafo	Perspectivas individuales de agentes específicos

4. Configuración de Credenciales

 **Archivo de credenciales:** Crea el archivo `backend/.env` basándote en el template `backend/.env.example`. **Nunca subas el archivo .env al repositorio.** Ya está incluido en `.gitignore`.

Archivo: `backend/.env`

```
backend/.env

# ===== # CONFIGURACIÓN LLM PRINCIPAL
(requerido) # Soporta cualquier API con formato OpenAI SDK # Recomendado: Alibaba Qwen -
https://bailian.console.aliyun.com/ #

===== LLM_API_KEY=sk-
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX LLM_BASE_URL=https://dashscope.aliyuncs.com/compatible-
mode/v1 LLM_MODEL_NAME=qwen-plus #

===== # ZEP CLOUD - Knowledge Graph
(requerido) # Registro gratuito: https://app.getzep.com/ #

=====
ZEP_API_KEY=z_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX #

===== # LLM BOOST (opcional) - LLM
secundario más rápido/potente # Si no se usa, NO incluir estas variables en el .env #

===== LLM_BOOST_API_KEY=sk-
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX LLM_BOOST_BASE_URL=https://api.openai.com/v1
LLM_BOOST_MODEL_NAME=gpt-4o
```

Dónde Obtener Cada Credencial

Variable	Proveedor	URL de Registro	¿Requerido?
<code>LLM_API_KEY</code>	Alibaba Bailian / OpenAI / cualquier compatible	bailian.console.aliyun.com	Requerido
<code>LLM_BASE_URL</code>	URL del endpoint del proveedor LLM	Según proveedor elegido	Requerido
<code>LLM_MODEL_NAME</code>	Nombre del modelo a usar	Según proveedor (ej: qwen-plus)	Requerido
<code>ZEP_API_KEY</code>	Zep Cloud	app.getzep.com	Requerido

LLM_BOOST_API_KEY

Cualquier proveedor LLM
alternativo

Según elección

Opcional

Configuración para OpenAI (alternativa)

● ● ● backend/.env – Configuración OpenAI

```
LLM_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx LLM_BASE_URL=https://api.openai.com/v1  
LLM_MODEL_NAME=gpt-4o-mini ZEP_API_KEY=z_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```



Si usas OpenAI y recibes error **429 (insufficient_quota)**, recarga créditos en platform.openai.com/settings/billing. Los errores de cuota son del proveedor, no del sistema.

5. Pipeline de 5 Pasos – Detalle Técnico



El pipeline es **secuencial**: cada paso produce outputs que son inputs del siguiente. El estado se persiste en disco, permitiendo retomar desde cualquier punto.

1

Generación de Ontología & Construcción del Grafo

OntologyGenerator + GraphBuilderService

¿Qué hace? El usuario sube uno o más documentos (PDF/MD/TXT) junto con una descripción del objetivo de la simulación. El sistema extrae el texto, lo analiza con LLM y genera una ontología estructurada con tipos de entidades y relaciones. Luego construye el Knowledge Graph en Zep Cloud.

POST /api/graph/ontology/generate → Genera ontología (10 tipos de entidad, 6-10 tipos de relación)

POST /api/graph/build → Construye el Knowledge Graph en Zep con la ontología

PyMuPDF

LLM (JSON mode)

Zep Cloud Graph

Chunking configurable

Outputs: `project.json` con texto extraído, ontología definida y `graph_id` de Zep.

2

Preparación del Entorno de Simulación

ZepEntityReader + OasisProfileGenerator + SimulationConfigGenerator

¿Qué hace? Lee las entidades del Knowledge Graph de Zep y convierte cada una en un perfil de agente OASIS con personalidad, historia y comportamientos detallados. Simultáneamente genera la configuración completa de la simulación usando LLM.

POST /api/simulation/prepare → Lee entidades de Zep + genera perfiles de agentes

Zep Entity Reader

OASIS Profiles

LLM (persona generation)

5 workers paralelos

Outputs: `reddit_profiles.json`, `twitter_profiles.csv`, `simulation_config.json`

3

Ejecución de la Simulación

SimulationRunner (subprocess OASIS/CAMEL-AI)

¿Qué hace? Lanza la simulación en un subprocess Python independiente. Los agentes interactúan en plataformas simuladas de Twitter y/o Reddit por N rondas. Cada acción (tweet, like, comment, follow) queda registrada.

Actualizaciones en tiempo real vía polling.

POST /api/simulation/start → Inicia subprocess de simulación

GET /api/simulation/run-state/{id} → Estado en tiempo real

POST /api/simulation/stop → Detiene la simulación

camel-oasis 0.2.5

camel-ai 0.2.78

LLM (agent decisions)

Twitter + Reddit

Outputs: `actions.jsonl` (log completo de acciones), `run_state.json` (estado de ejecución)

4

Generación del Reporte

ReportAgent – Patrón ReACT

¿Qué hace? El ReportAgent utiliza el patrón ReACT (Razonamiento → Acción → Observación) para generar un reporte multi-sección de análisis de la simulación. Llama herramientas de Zep para recuperar contexto relevante, analiza los logs de acciones y produce insights profundos con hasta 2 rondas de reflexión por sección.

```
POST /api/report/generate → Genera reporte asíncrono con ReportAgent
```

```
GET /api/report/{id} → Recupera el reporte generado (texto + PDF)
```

ReACT Pattern

LLM (temp=0.5)

InsightForge + PanoramaSearch

Max 5 tool calls

Outputs: `report.md`, `report.pdf` (HTML corporativo), `agent_log.jsonl` (trazabilidad completa del razonamiento)

5

Interacción Profunda

ReportAgent – Modo Chat

¿Qué hace? Permite al usuario hacer preguntas de seguimiento al ReportAgent. El agente mantiene contexto del reporte generado y puede recuperar información adicional del Knowledge Graph de Zep para responder preguntas específicas sobre la simulación.

```
POST /api/report/{id}/chat → Envía mensaje al ReportAgent
```

Chat contextual

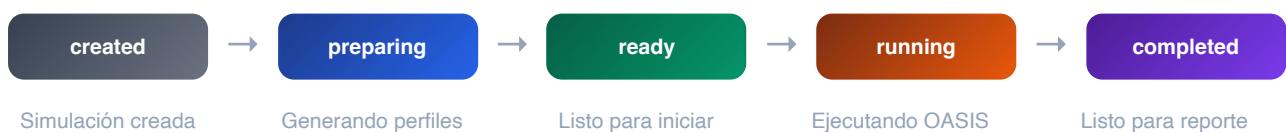
Zep hybrid search

LLM conversational

Historial persistente

Outputs: Respuestas en tiempo real con razonamiento trazable. El historial de conversación se persiste con el reporte.

Diagrama de Estado de la Simulación



6. Stack Tecnológico Completo

Backend — Python

Paquete	Versión	Uso en el Proyecto
<code>flask</code>	≥3.0.0	Framework web REST API — servidor principal en puerto 5001
<code>flask-cors</code>	≥6.0.0	Habilita CORS para que el frontend Vue pueda comunicarse
<code>openai</code>	≥1.0.0	SDK unificado para todos los proveedores LLM
<code>zep-cloud</code>	3.13.0	Knowledge Graph: almacenamiento y consulta de entidades
<code>camel-oasis</code>	0.2.5	Framework OASIS para simulación de redes sociales
<code>camel-ai</code>	0.2.78	Framework multi-agente base para OASIS
<code>torch</code>	≥2.0.0, ≤2.2.2	Deep learning requerido por CAMEL-AI (macOS Intel: ≤2.2.2)
<code>PyMuPDF</code>	≥1.24.0	Extracción de texto de documentos PDF
<code>python-dotenv</code>	≥1.0.0	Carga variables de entorno desde archivo .env
<code>pydantic</code>	≥2.0.0	Validación y serialización de modelos de datos
<code>charset-normalizer</code> / <code>chardet</code>	≥3.0.0 / ≥5.0.0	Detección de encoding para archivos de texto no-UTF8

Frontend — JavaScript / Vue

Paquete	Versión	Uso en el Proyecto
<code>vue</code>	^3.5.24	Framework de UI — Composition API con <script setup>
<code>vue-router</code>	^4.6.3	Navegación SPA entre las 5 vistas del pipeline
<code>vue-i18n</code>	^11.3.0	Internacionalización — Español, English, 中文
<code>axios</code>	^1.14.0	Cliente HTTP para comunicación con el backend Flask

`d3`

^7.9.0

Visualización del Knowledge Graph (nodos y aristas)

`vite`

^5.4.21

Build tool y servidor de desarrollo (HMR)

Herramientas de Desarrollo



Gestión de Dependencias Python

`uv` — Gestor de paquetes Python ultrarrápido.
Reemplaza pip + virtualenv. Los comandos del proyecto usan `uv run python ...`



Gestor de Paquetes Node

`npm` — Gestor estándar para el frontend Vue.
Comandos: `npm install` y `npm run dev`

7. Estructura de Archivos del Proyecto

```
MiroFish/ ← Raíz del proyecto | — backend/ | — .env ← CREDENCIALES (no commitear) | —  
.env.example ← Template de credenciales | — pyproject.toml ← Dependencias Python (uv) |  
| — run.py ← Entry point Flask (puerto 5001) | — app/ | — __init__.py ← Factory Flask  
+ blueprints | — api/ | | — graph.py ← Endpoints /api/graph/* | | — simulation.py ←  
Endpoints /api/simulation/* | | — report.py ← Endpoints /api/report/* | — services/ ←  
Lógica de negocio principal | | — ontology_generator.py ← Step 1: Extracción de  
ontología | | — graph_builder.py ← Step 1: Construcción Knowledge Graph | | —  
zep_entity_reader.py ← Step 2: Lectura de entidades Zep | | — oasis_profile_generator.py  
← Step 2: Perfiles de agentes | | — simulation_config_generator.py ← Step 2: Config  
simulación | | — simulation_runner.py ← Step 3: Ejecución OASIS | | — report_agent.py  
← Step 4-5: ReportAgent ReACT | | — pdf_generator.py ← Generación de PDF corporativo | |  
| — simulation_manager.py ← Gestión de ciclo de vida simulación | | — zep_tools.py ←  
Herramientas Zep del ReportAgent | | — zep_graph_memory_updater.py ← Actualización grafo  
post-sim | — utils/ | — llm_client.py ← Cliente LLM unificado (retry, JSON) | —  
file_parser.py ← Parser PDF/MD/TXT | — frontend/ | — package.json ← Dependencias npm |  
| — index.html ← Entry point HTML (favicon, título) | — public/ | | — MinkOracle.png ←  
Logo oficial (SVG original en mink_logo.svg) | — src/ | — views/ ← Vistas de cada paso  
| | — Home.vue ← Landing page | | — Process.vue ← Steps 1-2 (grafo + preparación) | |  
| — SimulationRunView.vue ← Step 3 (simulación) | | — ReportView.vue ← Step 4 (reporte)  
| | — InteractionView.vue ← Step 5 (chat) | — components/ | — Step1GraphBuild.vue ←  
Upload + ontología | — Step2EnvSetup.vue ← Agentes + config | — Step3Simulation.vue ←  
Monitor simulación | — Step4Report.vue ← Viewer + export PDF | — Step5Interaction.vue  
← Chat con agente | — locales/ ← i18n: en.json / es.json / zh.json
```


8. API REST – Referencia de Endpoints

Blueprint: Graph (/api/graph)

Método	Endpoint	Descripción
POST	/api/graph/upload	Sube documentos al proyecto
POST	/api/graph/ontology/generate	Genera ontología con LLM desde documentos
POST	/api/graph/build	Construye Knowledge Graph en Zep
GET	/api/graph/data/{graph_id}	Obtiene nodos y aristas para visualización D3
GET	/api/graph/project/{project_id}	Datos del proyecto (metadata, graph_id)
GET	/api/graph/history	Lista todos los proyectos históricos

Blueprint: Simulation (/api/simulation)

Método	Endpoint	Descripción
POST	/api/simulation/create	Crea nueva simulación para un proyecto
POST	/api/simulation/prepare	Genera perfiles de agentes + config desde Zep
POST	/api/simulation/start	Inicia el subprocess de simulación OASIS
POST	/api/simulation/stop	Detiene la simulación en curso
POST	/api/simulation/close-env	Cierra el entorno de simulación limpiamente
GET	/api/simulation/{id}	Estado y metadata de la simulación
GET	/api/simulation/run-state/{id}	Estado en tiempo real (rondas, acciones, agentes)
GET	/api/simulation/config/{id}	Configuración de tiempos y parámetros
GET	/api/simulation/env-status	Estado del entorno de simulación (alive/dead)

Blueprint: Report (/api/report)

Método	Endpoint	Descripción
POST	/api/report/generate	Inicia generación asíncrona del reporte con ReportAgent
GET	/api/report/{id}	Obtiene el reporte generado (secciones, metadata)
GET	/api/report/{id}/pdf	Descarga el PDF corporativo del reporte
POST	/api/report/{id}/chat	Envía mensaje al ReportAgent (chat interactivo)
GET	/api/report/by-simulation/{sim_id}	Obtiene reporte asociado a una simulación



Todas las respuestas siguen el formato: `{"success": true/false, "data": {...}, "error": "..."} . El backend corre en http://localhost:5001 y el frontend en http://localhost:3000 .`

9. Comandos para Ejecutar el Proyecto

Requisitos Previos

- **Python 3.11+** — verificar con `python3 --version`
- **Node.js 18+** — verificar con `node --version`
- **uv** — gestor de paquetes Python: `pip install uv`
- **Archivo** `backend/.env` — con las credenciales configuradas (ver Sección 4)

Instalación (Primera vez)

Terminal – Instalación inicial

```
# 1. Clonar el repositorio git clone https://github.com/tatanGod17/mirofish.git cd mirofish
# 2. Instalar dependencias del backend cd backend uv sync cd .. # 3. Instalar dependencias
del frontend cd frontend npm install cd .. # 4. Crear archivo de credenciales cp
backend/.env.example backend/.env # → Editar backend/.env con tus API keys
```

Ejecución del Proyecto

Terminal 1 – Backend Flask (puerto 5001)

```
cd backend uv run python run.py # → http://localhost:5001
```

Terminal 2 – Frontend Vue (puerto 3000)

```
cd frontend npm run dev # → http://localhost:3000
```

Comandos Útiles

Comando	Descripción
<code>lsof -i:5001 xargs kill -9</code>	Matar proceso en puerto 5001 (backend)
<code>lsof -i:3000 xargs kill -9</code>	Matar proceso en puerto 3000 (frontend)
<code>cd backend && uv add <paquete></code>	Agregar nueva dependencia Python
<code>cd frontend && npm install <paquete></code>	Agregar nueva dependencia npm

```
cd backend && uv run python -m pytest
```

Ejecutar tests del backend



macOS con chip Intel: torch tiene restricción de versión `≤2.2.2` para compatibilidad. En Apple Silicon (M1/M2/M3), no hay restricción de versión.

10. Repositorio & Recursos



Repositorio Principal

<https://github.com/tatanGod17/mirofish>

Repositorio público bajo licencia AGPL-3.0. Incluye backend Flask, frontend Vue 3, locales y documentación.

Links de Servicios Externos

Servicio	URL	Para qué
Alibaba Bailian	bailian.console.aliyun.com	Obtener LLM_API_KEY (Qwen) — proveedor recomendado
OpenAI Platform	platform.openai.com	Alternativa LLM (GPT-4o, GPT-4o-mini)
Zep Cloud	app.getzep.com	Obtener ZEP_API_KEY — Knowledge Graph
OASIS Framework	github.com/camel-ai/oasis	Documentación del motor de simulación
CAMEL-AI	github.com/camel-ai/camel	Framework multi-agente base

Datos de Archivos Clave

Archivo	Tamaño	Descripción
report_agent.py	~99 KB	Componente AI central — ReportAgent con patrón ReACT
simulation_runner.py	~69 KB	Motor de ejecución OASIS con tracking en tiempo real
oasis_profile_generator.py	~49 KB	Generador de perfiles de agentes desde entidades Zep
simulation_config_generator.py	~39 KB	Generador LLM de configuración completa de simulación
Step4Report.vue	~147 KB	Componente frontend más grande — visor y exportador de reportes



Tier Gratuito Soportado



Recomendaciones de Uso

- Zep Cloud free tier para grafos pequeños-medianos
- Alibaba Qwen free credits para primeras pruebas

- Comenzar con simulaciones de <40 rondas
- Monitorear consumo de tokens LLM en el dashboard del proveedor

MinkOracle v0.1-Preview

Motor de Inteligencia de Enjambre Conciso y Universal

github.com/tatanGod17/mirofish · Licencia AGPL-3.0